



Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Licenciatura en Informática

Nombre del alumno

Daniel Gerardo Reyes Castro

MATERIA

PROGRAMACION DE DISPOSITIVOS MOVILES

Nombre del Asesor

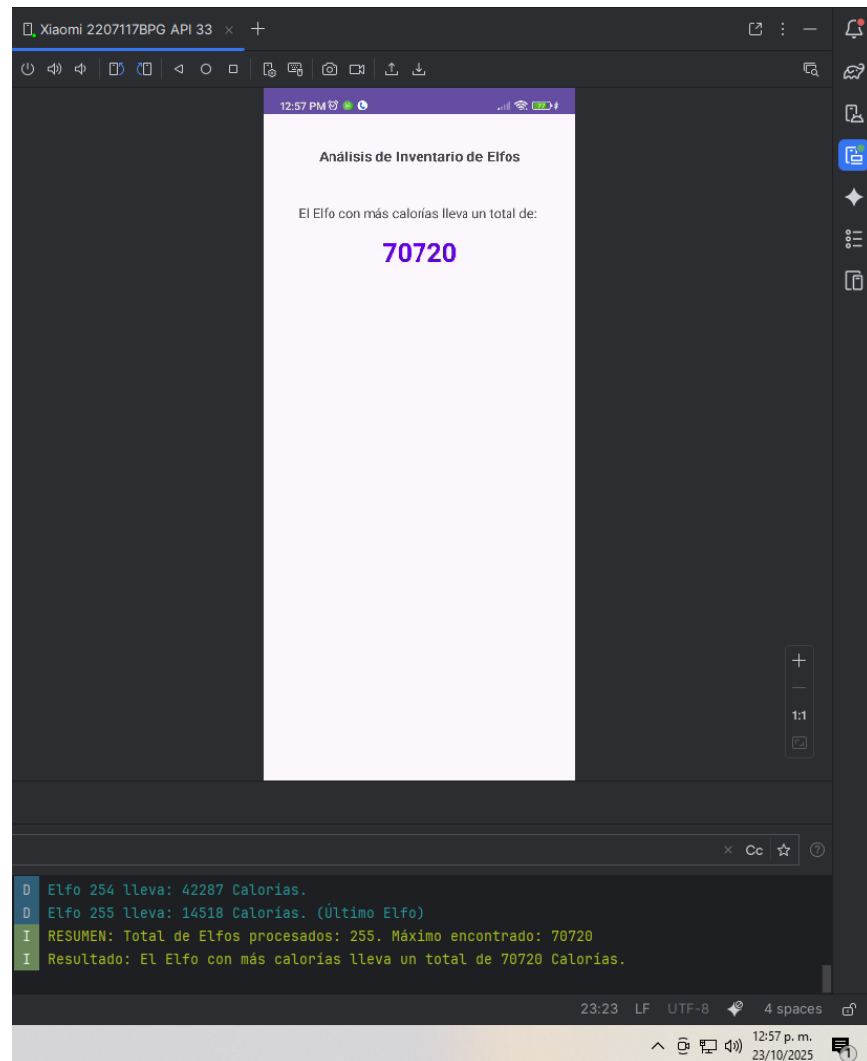
CRISTIAN CARDOSO ARELLANO

UNIDAD 5

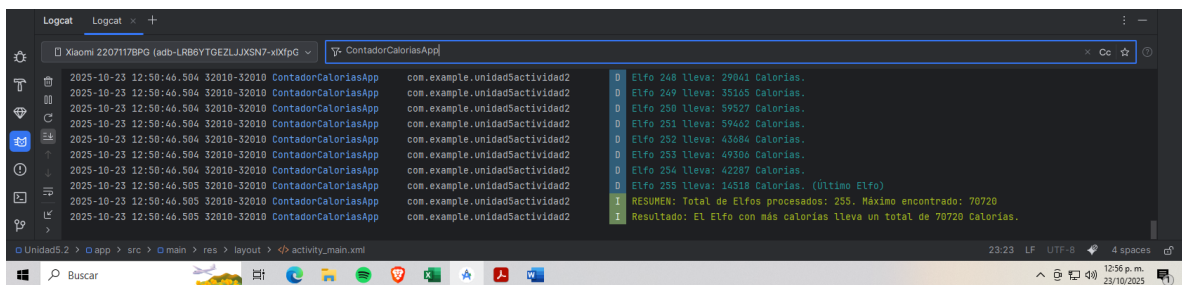
Conteo de calorías

ACTIVIDAD 2

Captura de funcionamiento de la app



Captura de ventana del Logcat



Código fuente

```
package com.example.unidad5actividad2;

import androidx.appcompat.app.AppCompatActivity;
```

```

import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
import android.content.res.AssetManager;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.BufferedReader;

public class MainActivity extends AppCompatActivity {

    private static final String TAG = "ContadorCaloriasApp";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Referencia al TextView
        TextView maxCaloriasTextView =
            findViewById(R.id.textViewMaxCalorias);

        long maxCalorias = MaxCalorias("calorias.txt");

        // Muestra el resultado final en logcat
        if (maxCalorias > 0) {
            Log.i(TAG, "Resultado: El Elfo con más calorías lleva un
total de " + maxCalorias + " Calorías.");
            maxCaloriasTextView.setText(String.valueOf(maxCalorias));
        } else {
            maxCaloriasTextView.setText("ERROR: No se pudieron procesar
los datos.");
            Log.e(TAG, "No se pudieron obtener los datos del archivo
calorias o el resultado es cero.");
        }
    }

    private long MaxCalorias(String fileName) {

        AssetManager assetManager = getAssets();
        InputStream inputStream = null;
        BufferedReader reader = null;

        long maxCalorias = 0;
        long currentElfTotal = 0;
        int elfIndex = 0;

        try {
            inputStream = assetManager.open(fileName);
            // UTF-8 para evitar problemas de codificación
            reader = new BufferedReader(new
InputStreamReader(inputStream, "UTF-8"));
            String line;
            Log.d(TAG, "Iniciando lectura de Assets");

            while ((line = reader.readLine()) != null) {

```

```

        // Micro-optimización CRUCIAL para eliminar el carácter
        \r que deja Windows
        String trimmedLine = line.replace('\r', ' ').trim();
        // Si la línea está vacía o solo contiene espacios en
        blanco, significa FIN DE INVENTARIO
        if (trimmedLine.isEmpty()) {
            // Solo si el Elfo tenía calorías, lo contamos y
            procesamos.
            if (currentElfTotal > 0) {
                elfIndex++;
                Log.d(TAG, "Elfo " + elfIndex + " lleva: " +
                currentElfTotal + " Calorías.");
                if (currentElfTotal > maxCalorias) {
                    maxCalorias = currentElfTotal;
                    Log.i(TAG, "Nuevo Máximo Global en Elfo #" +
                    elfIndex + ". Total: " + maxCalorias);
                }
            }
            currentElfTotal = 0; // Reinicia el contador para el
            siguiente Elfo
        } else {
            // Es un ítem de comida, súmalo al Elfo actual
            try {
                long calories = Long.parseLong(trimmedLine);
                currentElfTotal += calories;
            } catch (NumberFormatException e) {
                Log.e(TAG, "Error al parsear línea: " +
                trimmedLine);
            }
        }
    }

    // Procesar el último Elfo (si el archivo termina sin una
    línea en blanco)
    if (currentElfTotal > 0) {
        elfIndex++;
        Log.d(TAG, "Elfo " + elfIndex + " lleva: " +
        currentElfTotal + " Calorías. (Último Elfo)");
        if (currentElfTotal > maxCalorias) {
            maxCalorias = currentElfTotal;
        }
    }
    Log.i(TAG, "RESUMEN: Total de Elfos procesados: " + elfIndex
    + ". Máximo encontrado: " + maxCalorias);

} catch (IOException e) {
    Log.e(TAG, "Error al leer Asset File: " + fileName, e);
    return 0;
} finally {
    try {
        if (reader != null) reader.close();
        if (inputStream != null) inputStream.close();
    } catch (IOException e) {
        Log.e(TAG, "Error al cerrar flujos.", e);
    }
}
return maxCalorias;

```

```
}  
}
```

Conclusión

El objetivo principal de encontrar la suma máxima de calorías por Elfo representó todo un reto para mí ya que no lograba hacer que se tomara el archivo de calorías completo y no leía a todos los Elfos.

Tuve la necesidad de intentar compilar los recursos por diferentes métodos usando `R.string` o `res/raw` para archivos grandes. La implementación utilizando `AssetManager` con un `BufferedReader` garantizó la lectura eficiente de los más de 2200 elementos del input. Analicé también el problema de compatibilidad de los saltos de línea (`\r\n` de Windows), aplicando la corrección en el código usando `.replace("\r", '')` para que el algoritmo de separación de Elfos (`\n\n`) funcionara de manera correcta.